

SIT725

8.2HD



DOCKER: END-TO-END APPLICATION DEPLOYMENT

Intro

This task is required only if you are aiming to claim a High Distinction (HD) in SIT725.

In this task, you will individually dockerise your group project application. Even though your web application is a group project, this Docker task is an individual deliverable.

Each group member who wishes to target an HD **must complete this task on their own**, by:

1. Cloning or forking the group project repository into their own GitHub account.
2. Containerising the entire application via Docker, ensuring it runs end-to-end.
3. Embedding their student ID into the app (see below).
4. Writing a brief personal reflection (see below).
5. Submitting their own evidence and repo individually.

This means **every HD-seeking group member dockerises the same group project app** but must do so independently and in their own repository.

Instructions

1. Learn Docker: Attend the workshop session and learn how to use Docker to “Distribute” your application.

You can also use the video below:

<https://youtu.be/0EJu1ienLdM>

2. Dockerise Your App:

You must dockerise your application such that it runs correctly in a clean environment. Make sure:

- The server runs correctly inside the containerised environment
- All required dependencies are installed and available at runtime
- Database integration is fully functional
- The application works as expected when accessed via localhost after being started in the containerised environment.

Containerising only the backend is not sufficient.

This task requires that the entire application functions end-to-end, including any database-backed features. You must use Docker to containerise your application.

The specific configuration and setup decisions (e.g. how services are connected and configured at runtime) are part of the assessment and are left to you.

3. Embed Your Identity in the App To uniquely identify your submission:

- Add a new REST API route `/api/student` in your Node.js server.
- This route must return a JSON object with your full name and student ID.

Example:

```
{
  "name": "Jane Doe",
  "studentId": "123456789"
}
```

- The endpoint should be accessible from the Dockerised version of your app at:

<http://localhost:<your-port>/api/student>

4. Reflection:

Write a concise (max 500 words) reflection covering:

- The specific tooling and configuration approaches you chose to containerise the application (e.g. how you handled dependencies, services, and runtime configuration), and why you made these choices.
- Any issues you encountered during the process and how you resolved them
- What you found most challenging or rewarding about the task

5. Document How to Run Your Container

You must include clear, complete instructions in your main README.md that allow a marker to run your application from scratch.

Your instructions must include:

- How to build and start the application
- Any required setup steps
- Which port to access via localhost
- What the `/api/student` endpoint should return

Markers will follow only your README instructions.

Configuration and Sensitive Information

Your application may require **runtime configuration values** that should not be committed to a public repository.

You must ensure that:

- Sensitive configuration values are **not hard-coded or publicly exposed**
- The marker is still able to **run and verify your application successfully**
- Your README clearly explains **how the marker should obtain or provide the required configuration values**

If certain configuration values are intentionally excluded from the repository for security reasons, you must:

- Clearly state this in your README
- Explain **where the marker can find the required information** to run the application (e.g. via your OnTrack submission)
- Ensure the steps required to run the application are **unambiguous and complete**

Markers should not have to guess missing information or modify your application to make it run.

Failure to clearly document how required configuration is supplied may result in the submission **not meeting HD requirements**, even if the application works on your own machine.

Note:

As part of marking, we will verify that the application's database-dependent functionality works correctly, amongst any other dependencies (if applicable).

For example, this could include, but is not limited to:

- User signup
- User login
- Any other core feature that requires database access

The specific feature used to demonstrate database integration will depend on your group project.

If the database is not correctly connected or functioning, the submission will not meet HD requirements, even if the server starts successfully.

As per unit policy, this HD task allows only **ONE round of feedback.**

Therefore, it is strongly recommended that before submitting, you thoroughly **test your containerised application** by running it in a fresh environment (e.g. a VM or another machine).

Submission details and Delivery

- Once you are done, push all required code and documentation to your own public GitHub repository. Pls follow the GitHub guidelines given in Task 1.3P.
- Ensure your repository includes:
 - All containerization-related configuration
 - Updated server code with `/api/student`
 - Clear run instructions in README.md
- Take screenshots showing:
 - The application running
 - The `/api/student` endpoint response
 - Evidence of working functionality
- Combine screenshots and your reflection into a single PDF
- Submit the PDF and your GitHub repository link via OnTrack

If you want to be sure, simply clone your repo. If you can clone it, I can clone it 😊